

Do you know about the Akinator guessing game? Can you explain me the algorithm behind the accurate guess? How was the system was designed the frontend, backend and full database system? How can I rebuild this guessing game for my own Bangla language? Please explain me in details.

The **Akinator** game feels like pure magic, but under the hood, it is a masterclass in **Information Theory, Probability, and Expert Systems**.

## Part 1: The Algorithm Behind Akinator's Accurate Guesses

Akinator does not use a rigid, static flowchart. Instead, it runs on a dynamic **Probabilistic Expert System** (often powered by an engine named *Limule*). Here is how the core math and logic work:

1. **The Feature Matrix (Characters × Questions):**
   Imagine a massive table where rows are characters (e.g., *Humayun Ahmed*, *Shakib Al Hasan*, *Batman*) and columns are questions (e.g., *Is the character fictional?*, *Are they a cricketer?*). Each cell holds a probability weight representing how likely that character is to possess that attribute (Yes, No, Don't Know, or Partially).

2. **Maximum Information Gain (Shannon Entropy):**
   After every answer you give, the algorithm calculates which remaining question will split the remaining pool of characters as close to **50/50** as possible. In information theory, this is called maximizing **Information Gain** or reducing **Entropy**.

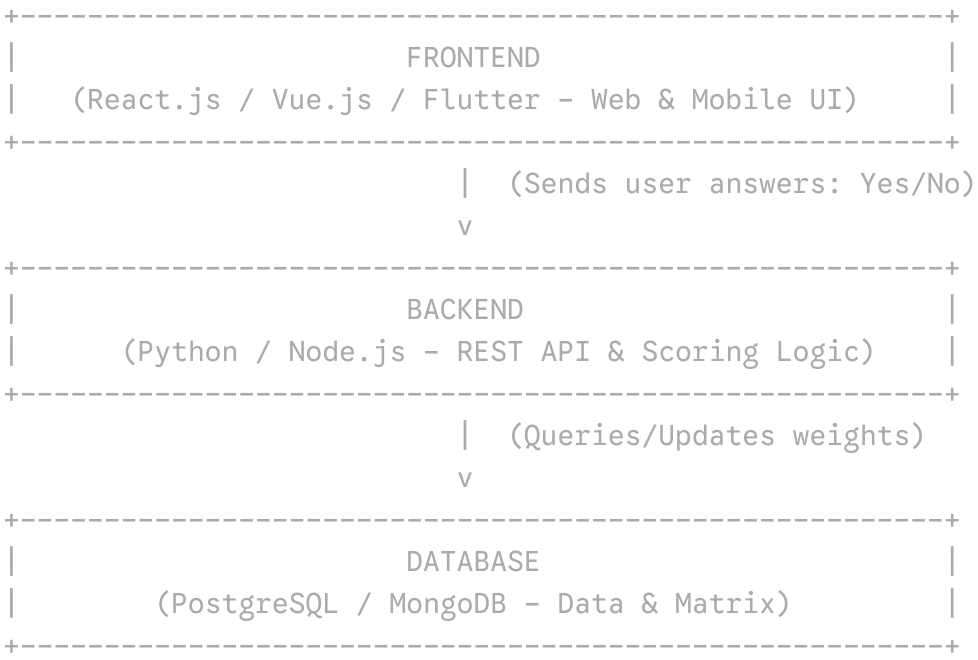
   > If there are 1,000 characters left, and Question A divides them into 500 Yes and 500 No, it is a great question.
   >
   > If Question B divides them into 990 Yes and 10 No, it is a terrible question because it doesn't eliminate many possibilities.

3. **Probability Scoring:**
   Every character starts with a baseline probability score. As you answer questions, the backend recalculates the probability for every single character in the database using a variant of **Bayes' Theorem**. Characters that match your answers rise to the top; mismatching characters drop down.

4. **Self-Learning Mechanism:**
   When Akinator fails to guess your character, it asks you for the character's name and a few distinguishing questions. It immediately injects this new data into its database, making it smarter for the next player.

## Part 2: System Architecture Design

Building a clone of Akinator requires a clean separation of concerns across the frontend, backend, and database.




### 1. Frontend (The User Interface)

**Tech Stack:** React.js, Vue.js, or simple HTML5/CSS/JavaScript for web; Flutter or React Native for mobile.

**Responsibilities:**

Displays the Genie character with animations.

Renders the current question text clearly.

Provides intuitive response buttons: **Yes**, **No**, **Don't Know**, **Probably**, **Probably Not**.

Shows progress bars or loading states while the backend processes the next move.

## 2. Backend (The Game Engine)

**Tech Stack:** Python (using **FastAPI** or **Flask** because of Python's superior data-handling libraries) or Node.js (Express).

**Responsibilities:**

Manages active user sessions (tracking which question number they are on and which characters have been filtered).

Executes the probability and entropy calculation algorithm to pick the next best question.

Evaluates when a character's probability crosses a high confidence threshold (e.g., > 98%) to trigger a "Guess" screen.

## 3. Database System

**Tech Stack:** Relational database like **PostgreSQL** or a NoSQL database like **MongoDB**.

**Core Tables/Collections:**

`characters` **Table:** `id`, `name`, `category` (e.g., Actor, Historical, Anime), `image_url`, `times_guessed`.

`questions` **Table:** `id`, `question_text_bn` (Bangla text), `weight_multiplier`.

`character_answers` **Mapping Table:** Links `character_id` and `question_id` with an answer value (1 for Yes, 0 for No, 0.5 for Don't Know).

## Part 3: How to Build Your Own Bangla Akinator

If you want to build a localized **Bangla Akinator** (targeting Bangladeshi pop culture, writers, historical icons, cricketers, and local internet culture), follow this blueprint:

## Step 1: Bootstrap the Data (Bangla Context)

Start small. Don't aim for 100,000 characters on day one. Create a seed dataset of **50 popular Bangla personalities** (e.g., Humayun Ahmed, Rabindranath Tagore, Kazi Nazrul

Islam, Shakib Al Hasan, Mosharraf Karim, Tahsan, etc.) and **20 core questions** translated into natural Bangla (e.g., চরিত্রটি কি বাস্তব মানুষ?, সে কি কোনো ক্রিকেটার?, সে কি লেখালেখির সাথে যুক্ত?).

## Step 2: Set Up the Database (PostgreSQL Example)

Create a structure to store characters and how they relate to questions:

SQL

```sql
-- Characters table
CREATE TABLE characters (
    id SERIAL PRIMARY KEY,
    name_bn VARCHAR(255) NOT NULL,
    image_url TEXT
);

-- Questions table
CREATE TABLE questions (
    id SERIAL PRIMARY KEY,
    text_bn TEXT NOT NULL
);

-- Matrix table for answers
CREATE TABLE character_question_matrix (
    character_id INT REFERENCES characters(id),
    question_id INT REFERENCES questions(id),
    answer_value FLOAT -- 1.0 for Yes, 0.0 for No, 0.5 for Don't Know
);
```

## Step 3: Implement the Core Algorithm (Python Backend)

You can write a lightweight scoring engine in Python. Here is a simplified logic snippet showing how probabilities are evaluated:

Python

```
def get_next_question(active_characters, questions, current_matrix):
  # Find the question that splits the remaining characters closest to 50%
  best_question = None
  min_score_diff = float("inf")

  for q in questions:
    yes_count = sum(
        1
        for c in active_characters
        if current_matrix.get((c, q), 0.5) == 1.0
    )
    # Calculate how close this question is to a 50/50 split
    diff = abs(len(active_characters) / 2 - yes_count)
    if diff < min_score_diff:
      min_score_diff = diff
      best_question = q

  return best_question
```

## Step 4: Build the Bangla Frontend

Use clean Bangla typography (such as Google Fonts **Noto Sans Bengali** or **SolaimanLipi**).

Set up a clean UI layout with large touch-friendly buttons for the responses: **হ্যাঁ (Yes)**, **না (No)**, **জানি না (Don't Know)**, **সম্ভবত (Probably)**.

Connect your frontend to your backend API endpoints (e.g., `/api/start-game`, `/api/answer`, `/api/guess`).

## Step 5: Implement the Learning Loop

Add a form at the end of the game: *"Did I fail? Tell me who you were thinking of."* When a user inputs a new Bangla character name and answers a few questions for it, write that data straight into your database so your custom Bangla Akinator grows smarter organically!

Ok, understand the system, now for the tech stack, I want to chose flutter for the frontend (in all round frontend setup in mobile, Android, iOS as well as in the web), spring boot for backend and postgresql for relational database sy

Your chosen tech stack—**Flutter (Frontend)** + **Spring Boot (Backend)** + **PostgreSQL (Database)**—is a **solid 8.5 / 10**.

It is an enterprise-grade, highly robust combination. You are pairing Google's premier cross-platform UI toolkit with Java's most powerful backend framework and one of the world's most reliable relational databases.

## Advantages of Your Stack

1. **True Code Reusability (Flutter):** Writing your UI once lets you deploy simultaneously to Android, iOS, and the Web. For a guessing game like Akinator, where the UI consists mostly of cards, text, buttons, and animations, Flutter handles this effortlessly.
2. **Enterprise Reliability (Spring Boot):** Spring Boot gives you production-ready features out of the box (security, robust dependency injection, easy REST API creation, and high concurrency handling). If your game goes viral and millions of users start playing at once, Spring Boot won't flinch.
3. **Data Integrity & Power (PostgreSQL):** PostgreSQL handles relational mapping brilliantly. Storing your characters, questions, and weight matrices using foreign keys and indexed lookups will keep query times lightning-fast.

## Disadvantages & Potential Friction Points

1. **Flutter Web Limitations:** While Flutter Web has matured significantly (using modern rendering like WebAssembly/CanvasKit), it still suffers from heavy initial bundle sizes (several megabytes) and poor SEO indexing. *(Note: For a game app, SEO doesn't matter much, but initial loading times can lag on slow mobile browsers compared to pure HTML/JS).*
2. **Java's Verbosity for Math/Algorithms:** Akinator's core engine relies heavily on probability calculations, scoring arrays, and matrix manipulation. While Java can do this, it requires more boilerplate code and manual logic implementation compared to data-centric languages like Python.

## Suggested Tweaks ("Something Better")

Depending on your priorities, here are two minor adjustments that could make your project even smoother:

### 1. Alternative Backend: Swap Spring Boot for FastAPI (Python)

**Why:** Because Akinator is fundamentally an **algorithmic and probability-driven game**, Python is the natural king of data logic. Using **Python with FastAPI** allows you to leverage libraries like `NumPy` or `Pandas` to compute entropy, information gain, and character probabilities in a fraction of the code lines Java requires. FastAPI is also lightning-fast and generates automatic API documentation (Swagger).

*Stick with Spring Boot if:* You already love Java, want rigid enterprise typing, or plan to scale this into a massive multi-service platform later.

### 2. Hybrid Frontend Strategy (If Web Performance Matters)

**Why:** If you notice Flutter Web feels a bit heavy or sluggish on low-end mobile browsers during testing, consider using **Flutter exclusively for Android and iOS**, and build a lightweight web version using **Next.js / React** (or even plain HTML/JS) talking to the same backend API.

*Stick with Flutter Web if:* Code reuse across all three platforms is your highest priority and you want to maintain a single codebase.

Ok, understood! Now I want to maintain a single codebase like flutter for frontend. And want to sawp into python from spring boot, but here is small problem that I'm familiar with spring boot java backend but no prior knowledge

Transitioning from **Spring Boot (Java)** to **Python** for an algorithmic game like Akinator is a fantastic choice. Don't worry about having no prior Python experience—your background in

Spring Boot actually gives you a huge head start because you already understand enterprise architecture, REST APIs, dependency injection concepts, and database mapping.

Here is a breakdown of how to tackle the Python backend, manage your AI tools, and choose between PostgreSQL and MongoDB.

## Part 1: Learning Python & Choosing the Right Framework

Since you are coming from Java, you should choose **FastAPI** as your Python backend framework.

### Why FastAPI is ideal for you:

1. **Type Hints (Feels like Java):** Unlike older Python frameworks (like Flask), FastAPI relies heavily on Python's modern type hints. It uses a library called **Pydantic**, which lets you define data models (similar to Java DTOs or classes with strict types). This makes the code much easier for you to read, write, and audit.
2. **Automatic Documentation:** The moment you write an endpoint, FastAPI automatically generates interactive Swagger documentation ( `/docs` ), letting you test your API straight from the browser without needing Postman.
3. **AI-Friendly:** Python is the native language of AI. If you ever want to integrate machine learning models or advanced math libraries later, FastAPI integrates with them seamlessly.

### How to review AI-generated Python code safely:

When you ask an AI to write your FastAPI code, look for these core elements to ensure it's correct:

**Imports:** Make sure it imports `FastAPI` , `BaseModel` (from Pydantic), and database connectors correctly.

**Asynchronous functions ( `async def` ):** FastAPI handles requests asynchronously. Ensure the AI isn't blocking the event loop with heavy synchronous database calls.

**Type validation:** Check that input data from the Flutter app is validated using Pydantic models (e.g., ensuring a user's answer is strictly a float between `0.0` and `1.0` ).

## Part 2: PostgreSQL vs. MongoDB for Akinator

Both databases can handle this game, but **MongoDB has a distinct edge** for an Akinator-style probability engine. Here is why:

## Option A: MongoDB (Recommended for this specific game)

Akinator relies on a **feature vector** (a list of questions and how a character answers them). In MongoDB, you can store a character as a single flexible JSON document containing an embedded map of answers:

JSON

```
{
  "name_bn": "হুমায়ূন আহমেদ",
  "category": "Writer",
  "answers": {
    "q1": 1.0,
    "q2": 0.0,
    "q3": 1.0
  }
}
```

**Advantage:** When your Python backend calculates probabilities or entropy, having the character data stored as a document structure maps directly to Python dictionaries and NumPy arrays without complex SQL `JOIN` queries. It makes reading and updating character weights incredibly fast and intuitive.

## Option B: PostgreSQL (Great if you prefer strict schemas)

**Advantage:** If you want strict relational integrity (ensuring a question ID always exists before a character answers it), PostgreSQL with tables and foreign keys is bulletproof.

**The Catch:** Calculating matrix weights across multiple tables ( `characters` , `questions` , and `character_question_matrix` ) requires writing multi-table `JOIN` queries, which can become slightly more verbose as your database grows.

## Final Verdict on Database:

Go with **MongoDB** if you want a natural, document-based layout that mimics how the AI processes character attributes. Go with **PostgreSQL** if you want to stick strictly to relational tables. For a beginner learning Python, MongoDB's document model often feels very smooth when paired with Python dictionaries.

> Ok, now I'm choosing the flutter frontend, python backend and mongodb for nosql database. Now please give me the full file directory mapping for every tech stack in flowcharts for my understanding to make the build plan for that Akinator game app.

Here is the complete, production-ready file directory mapping for your **Flutter (Frontend) + Python FastAPI (Backend) + MongoDB (Database)** stack. This structure is tailored specifically for your Bangla Akinator game.

## 1. Flutter Frontend Directory Mapping ( `akinator_frontend/` )

This structure separates UI screens, state management logic, network services, and models, keeping your code clean and maintainable.

Plaintext

```
akinator_frontend/
│
├── lib/
│   ├── core/
│   │   ├── constants.dart       # API base URLs, App colors, Bangla fonts (Solai
│   │   └── theme.dart           # App theme styling (Genie vibes, card designs)
│   │
│   ├── models/
│   │   ├── character_model.dart # Dart class for characters (id, name_bn, image_
│   │   ├── question_model.dart  # Dart class for questions (id, text_bn)
│   │   └── game_state_model.dart# Tracks current session, question count, probab
│   │
│   ├── services/
│   │   └── api_service.dart     # HTTP client (Dio or http) to talk to FastAPI b
│   │
│   ├── providers/
```

```
│   │   └── game_provider.dart   # State management (Riverpod/Provider) for game
│   │
│   ├── screens/
│   │   ├── home_screen.dart     # Welcome screen with "Start Game" button (শুরু ক
│   │   ├── game_screen.dart     # Main game loop showing questions and Yes/No bu
│   │   ├── guess_screen.dart    # Akinator makes a guess ("Is this your characte
│   │   ├── defeat_screen.dart   # Triggered if Akinator fails; asks user to inpu
│   │   └── add_character_screen.dart # Form to add a new Bangla character to DB
│   │
│   ├── widgets/
│   │   ├── custom_button.dart   # Styled answer buttons (হ্যাঁ, না, জানি না)
│   │   └── genie_avatar.dart    # Animated Genie image/widget component
│   │
│   └── main.dart                # App entry point
│
├── pubspec.yaml                 # Dependencies (dio, provider/riverpod, flutter_
└── .env                         # Environment variables (Backend API URL)
```

## 2. Python FastAPI Backend Directory Mapping ( `akinator_backend/` )

This structure uses FastAPI's modular design pattern. It separates your API routing from your core game algorithms (entropy calculation and probability scoring).

Plaintext

```
akinator_backend/
│
├── app/
│   ├── api/
│   │   ├── __init__.py
│   │   ├── endpoints.py         # API routers (/start-game, /answer, /guess, /ad
│   │   └── schemas.py           # Pydantic models for request/response validatio
│   │
│   ├── core/
│   │   ├── __init__.py
│   │   ├── config.py            # App configurations and environment settings
│   │   └── database.py          # MongoDB connection handler (using Motor or PyM
│   │
│   ├── services/
│   │   ├── __init__.py
│   │   └── game_engine.py       # Core AI logic: Shannon Entropy, Bayes' Theorem
```

```
│   │
│   └── main.py                  # FastAPI application entry point & middleware s
│
├── requirements.txt             # Python dependencies (fastapi, uvicorn, pymongo
├── .env                         # Environment variables (MongoDB Atlas URI, Data
└── README.md                    # Project documentation & run instructions
```

## 3. MongoDB Database Structure ( `bangla_akinator` )

Since MongoDB is a document store (NoSQL), your database collections organize data natively as JSON-like documents.

Plaintext

```
Database: bangla_akinator
│
├── Collection: `characters`
│   └── Document Example:
│       {
│         "_id": ObjectId("64a7..."),
│         "name_bn": "হুমায়ূন আহমেদ",
│         "category": "লেখক (Writer)",
│         "image_url": "https://...",
│         "answers": {
│           "q_01": 1.0,  // 1.0 = Yes, 0.0 = No, 0.5 = Don't Know
│           "q_02": 0.0,
│           "q_03": 1.0
│         }
│       }
│
└── Collection: `questions`
    └── Document Example:
        {
          "_id": ObjectId("64a8..."),
          "question_code": "q_01",
          "text_bn": "চরটি কি বাস্তব মানুষ?",
          "active": true
        }
```

## How to Execute Your Build Plan

1. **Step 1 (Database & Seed Data):** Set up a MongoDB cluster (locally or via MongoDB Atlas). Insert your initial collection of 20-50 Bangla personalities with hardcoded answer mappings.
2. **Step 2 (Backend Core):** Build `game_engine.py` in Python to fetch characters from MongoDB, calculate the next question using your entropy filter logic, and expose endpoints via FastAPI.
3. **Step 3 (Frontend UI):** Build the Flutter screens and connect `api_service.dart` to your FastAPI endpoints. Test the core loop (Home -> Question -> Answer -> Guess).
4. **Step 4 (Learning Loop):** Implement the "Defeat Screen" on Flutter and the `/add-character` endpoint on FastAPI so users can successfully expand your Bangla database organically!

> Ok, one more confusion, how about the data in the database? How to add them? How can I make this database with lot more data when anyone tries to play the game to find the right guess match with the system?

This is the **most crucial part** of building an Akinator-style game. When you first launch your game, you won't have 100,000 characters. You might only have 30 or 50.

Building a massive database manually is impossible. Instead, Akinator solved this using a brilliant strategy: **Crowdsourced Self-Learning**.

Here is how you populate your MongoDB database initially and how it scales up automatically as people play.

## Step 1: The Initial Seed (Bootstrapping Phase)

Before launching, you manually seed your MongoDB database with **50 to 100 well-known Bangla personalities** (actors, cricketers, writers, historical figures, politicians, and internet sensations) and around **20 universal core questions**.

In your MongoDB `characters` collection, a seed document will look like this:

JSON

```
{
  "name_bn": "সাকিব আল হাসান",
  "category": "ক্রিকেটার (Cricketer)",
  "image_url": "https://example.com/shakib.jpg",
  "answers": {
    "q1": 1.0,  // Is the character real? -> Yes
    "q2": 1.0,  // Are they related to sports/cricket? -> Yes
    "q3": 0.0   // Are they a writer? -> No
  }
}
```

## Step 2: How the Database Grows Automatically (The Magic Loop)

When your app goes live, users will think of characters that are **not** in your database yet. When this happens, the game enters its **Crowdsourced Learning Loop**:

1. **The Failure Trigger:** Akinator asks 20 questions, tries to make a guess, but all characters' probabilities drop low, or the user clicks *"None of these / My character is not here"* (defeat screen).
2. **The Capture Form:** Your Flutter app prompts the user:

   *"Ah, you stumped me! Who were you thinking of?"*

   User types the character's name in Bangla (e.g., "তাশরিফ খান" - Tashrif Khan) and optionally provides an image URL.
3. **Backfilling the Data (The Genius Part):**

   Instead of making the user answer *all* 20+ questions from scratch (which is annoying and causes users to quit), the backend looks at **the exact sequence of answers the user just gave during that game session**.

   The backend says: *"Ah, for this game session, the user answered Yes to 'Musician?', Yes to 'bangla band?', and No to 'Actor?'."*

   Your Python backend automatically creates a **new document** in MongoDB for "তাশরিফ খান" and maps those exact answers to him.

Plaintext

```
[User plays game] ──> [Akinator fails] ──> [User inputs: "Tashrif Khan"]
                                                        |
                                                        v
[New character auto-saved to MongoDB] <── [System binds user's previous Q&A histo
```

1. **Instant Scalability:** The very next time *another* user plays the game and answers similarly, your app will successfully guess "তাশরিফ খান". **Your database grows organically with every single loss!**

## Step 3: Handling Missing Answers (Data Sparsity)

As users add new characters, a character might only have answers for 5 out of your 25 total questions. What happens to the remaining 20 questions?

**Default to Neutral (0.5):** In your Python backend game engine ( `game_engine.py` ), if a character document doesn't have an entry for a specific question, code it to default to `0.5` (Don't Know / Neutral).

This ensures that missing data points don't break your math calculations or crash the entropy formula.

## Summary Checklist to Build Your Data Pipeline:

1. **Write a Python migration script** (or use MongoDB Compass GUI) to insert your initial 50 Bangla personalities.
2. **Build a** `/add-character` **FastAPI endpoint** that receives the character name and the session's historical question-answer mapping.
3. **Design the Defeat UI in Flutter** to gracefully capture the name and trigger that API endpoint whenever Akinator fails.

> Now I've a little concern about the user dissatisfaction about the app initially as the app can't give them right guess answer and the will get bad impration about using the app and then stop using the app! This will lead to a loss project! So How can I overcome this problem?

You have hit on one of the biggest challenges in software engineering and product design: **The Cold Start Problem**.

When you launch with a small database, users will naturally test the app with obscure characters, the app will fail, and they will uninstall it thinking it's broken.

To prevent your project from failing due to poor first impressions, you can use these **four battle-tested strategies** to mask the small database and turn failures into a feature rather than a bug.

## 1. Narrow the Launch Scope (Category-Specific Launch)

Instead of promising that your app can guess *any* Bangladeshi person in existence, **restrict the scope for Version 1.0**.

> **How it works:** Launch your app specifically as **"Bangla Cricketer Akinator"** or **"Bangla Novelists & Writers Akinator"**.
>
> **Why it works:** If you only have 50 characters, but *all 50 of them* are famous cricketers (Shakib, Mushfiq, Tamim, Mashrafe, Mustafizur, etc.), the app will guess correctly **95% of the time** during the first week. Once you have thousands of users and a growing database, you can open it up to actors, politicians, and general internet personalities.

## 2. Pre-Seed Heavily Before Launch (Don't launch with 30 items)

Spend a solid weekend before your public launch aggressively seeding your database. Aim for at least **150 to 200 core Bangla icons**:

> Top 40 Bangladeshi cricketers and footballers.
>
> Top 40 iconic writers, poets, and historical figures (Rabindranath, Nazrul, Humayun Ahmed, Begum Rokeya, Bangabandhu, etc.).

Top 40 popular actors, directors, and musicians (Mosharraf Karim, Chanchal Chowdhury, Tahsan, Arnob, etc.).

Top 40 global icons that everyone in Bangladesh knows (Messi, Ronaldo, Shah Rukh Khan, Marvel superheroes).
With 200 well-chosen characters, a new user's first 5 tries are almost guaranteed to succeed, hooking them instantly.

## 3. Gamify the Failure (Turn "Defeat" into a Celebration)

Never make the user feel like the app is *broken* when it fails. Make it feel like **they achieved something rare and impressive**.

**Change the UI Message:** Instead of showing a generic "Error: Character not found" screen, have the Genie look shocked, sweat, or throw its hands up in disbelief.

**Celebratory Copy:** *"Incredible! You beat the Genie! Only 2% of players manage to stump him today."*

**Why it works:** Humans love winning against AI. If failing feels like a fun challenge where *they* get to teach the AI something new, they won't get mad—they will feel proud and stay engaged.

## 4. Reward the "Teachers" (Crowdsourced Gamification)

When a user teaches your app a new character on the defeat screen, give them immediate social validation or in-game rewards.

**Badges & Titles:** Give them a fun title in the app like *"Master Teacher (ওস্তাদ)"* or award them points/coins for every character they successfully add to the database.

https://share.gemini.google/uXf0nej6nSFm
read this chat and please tell me where to start building and how can i open emty project files and then asigen AI agents to build the app for me?

I cannot directly browse or access external URLs like that shared Gemini link, so you will need to copy and paste the chat summary, requirements, or core ideas directly here.

Starting by creating empty files and asking an AI agent to "build the app for me" is a recipe for broken code. AI coding agents (such as Cursor Agent, Devin, Claude Code, or GitHub Copilot) are not autonomous lead engineers who can turn an empty folder into a production application from a single prompt. If you give an agent a blank slate with broad instructions, it will hallucinate dependencies, write monolithic files with conflicting patterns, and get stuck in execution loops.

Building with AI agents requires treating them like fast junior developers: you must scaffold the foundation first, set strict rules, and assign work in small, testable chunks.

## Step 1: Scaffold the Project (Never Start with Blank Files)

Do not manually create empty `.java`, `.ts`, or `.dart` files. Frameworks have strict directory hierarchies, build scripts, and dependency trees. Always use the framework's official CLI to generate the boilerplate.

Open your terminal (PowerShell or Bash) and generate the template for your chosen stack:

**For Angular:**

Bash

```
ng new my-app --routing --style=css
cd my-app
code .
```

**For Flutter:**

```bash
flutter create my_app
cd my_app
code .
```

**For Spring Boot (Backend):**
Use start.spring.io to download a Maven or Gradle zip with your required dependencies (e.g., Spring Web, Spring Data JPA, PostgreSQL Driver), extract it, and open that folder in your editor.

## Step 2: Establish the "Agent Contract" ( `AGENTS.md` )

Before talking to the agent, define the ground rules. Modern AI agent tools (Cursor, Claude Code, Copilot, Roo Code) automatically scan project instruction files at the root of the workspace.

Create a file named `AGENTS.md` (or `.cursorrules` if using Cursor) in your project root with the following format:

```markdown
# Project Guidelines

## Tech Stack
- Frontend: Angular 17+ (standalone components)
- Styling: Plain CSS
- State Management: RxJS Signals / Services

## Coding Rules
- Do not install new third-party libraries without explicit approval.
- Keep components small and split logic into services.
- Never write entire features into a single file.
- Write simple, beginner-friendly code with clear variable names and inline comme
- Always run the build/lint check before declaring a task complete.
```

### Step 3: Write a Concrete `SPEC.md`

Agents fail when requirements are vague. Create a `SPEC.md` file in the root folder listing your app's core data models, pages, and API endpoints:

Markdown

```
# App Specification: Task Tracker

## 1. Data Model
- Task: `id` (string), `title` (string), `completed` (boolean), `createdAt` (date

## 2. Views
- `/tasks`: List all tasks with a checkbox and delete button.
- Floating input at the top to add a new task.

## 3. Storage
- For now, persist tasks using browser `localStorage`.
```

### Step 4: Assign Work Slice-by-Slice

Open your agent interface (e.g., Cursor Composer in Agent mode, GitHub Copilot Chat in `@workspace` mode, or Claude Code) and prompt the agent using small, sequential tasks.

**Prompt 1 (Setup Data Model):**

> *"Read `AGENTS.md` and `SPEC.md`. Create the TypeScript interface for `Task` under `src/app/models/task.model.ts`. Do not touch any other files yet."*

**Prompt 2 (Service Layer):**

> *"Create a `TaskService` in `src/app/services/task.service.ts` that manages the task array using `localStorage`. Include methods for `getTasks()`, `addTask(title)`, `toggleTask(id)`, and `deleteTask(id)`. Keep the code simple."*

**Prompt 3 (UI Component):**

> *"Generate a standalone component for the task list. Connect it to `TaskService`. Display the list and wire up the 'add' input and 'delete' buttons. Run the app build command to verify there are no compilation errors."*

## How to Stay in Control

**Test every step locally:** Run your local development server ( `npm start` , `ng serve` , or `flutter run` ) after every prompt. Never prompt an agent for a new feature while the current code is broken.

**Review Git diffs:** Keep Git open. If an agent rewrites a file you did not ask it to change, revert that specific file immediately.

**Keep prompts narrow:** An agent that writes 30 lines of accurate code per prompt is far more useful than an agent attempting to generate 500 lines across 8 files at once.

## Want me to draft a custom AGENTS.md and project spec template for your tech stack?

↪ Yes

PDF

Building a Bangla Akinator...

I've uploaded the full chat here, now guid me which spacific software will need to setup in my pc to run the app into my local device and how to start building the project from scrach with the proper best practices?

For running your Bangla Akinator application locally on a Windows PC with your chosen stack—Flutter frontend, Python FastAPI backend, and MongoDB database—you will need a decoupled setup where the database, API server, and client run as independent local processes. PDF + 2

# 1. Software Required on Your PC

Install the following tools before generating any code:

**Python (3.11 or 3.12):** Download from python.org. Check the box **"Add python.exe to PATH"** during installation. This provides the runtime for FastAPI and Uvicorn. PDF + 1

**MongoDB Community Server & MongoDB Compass:** Download the Windows MSI installer from mongodb.com. Install it as a Windows Service (runs automatically on port `27017`). Ensure MongoDB Compass (the GUI viewer) is checked so you can visually inspect your characters and question collections. PDF + 1

**Flutter SDK (Stable):** Download the zip from docs.flutter.dev and extract it to a directory without spaces (e.g., `C:\src\flutter`). Add `C:\src\flutter\bin` to your Windows User `Path` environment variable.

**Android Studio / Google Chrome:**

For mobile testing: Install Android Studio, open the SDK Manager, download the Android SDK platform-tools, and configure an Android Virtual Device (AVD).

For web testing: Google Chrome is sufficient to run `flutter run -d chrome`.

**Git for Windows:** Download from git-scm.com for version control.

**Visual Studio Code:** Recommended extensions:

*Flutter* and *Dart*

*Python* and *Pylance*

*MongoDB for VS Code* (optional, for running database queries directly in the editor)

Run this verification check in PowerShell:

PowerShell

```
python --version
flutter doctor
```

```
mongod --version
```

Resolve any red checkmarks shown by `flutter doctor` (e.g., accepting Android licenses via `flutter doctor --android-licenses`).

## 2. Setting Up the Workspace Structure

Create a single workspace folder named `bangla_akinator` that isolates the frontend and backend: PDF + 1

Plaintext

```
bangla_akinator/
├── akinator_backend/
└── akinator_frontend/
```

Open PowerShell in the workspace directory:

PowerShell

```
mkdir bangla_akinator
cd bangla_akinator
```

## 3. Building Phase 1: Database & Seed Script

Do not create an empty database with no records. The game engine requires seed data to calculate probabilities and pick the first question. PDF + 1

### 1. Configure the Backend Directory

PowerShell

```
mkdir akinator_backend
cd akinator_backend
python -m venv venv
.\venv\Scripts\Activate.ps1
```

Create a `requirements.txt` file:

Plaintext

```
fastapi>=0.110.0
uvicorn[standard]>=0.28.0
motor>=3.3.0
pydantic>=2.6.0
python-dotenv>=1.0.0
```

Install them:

PowerShell

```
pip install -r requirements.txt
```

## 2. Scaffold the Backend Architecture

Set up the modular directory layout outlined in your blueprint: PDF

PowerShell

```
mkdir app
mkdir app\api, app\core, app\services, scripts
New-Item app\__init__.py, app\main.py, app\core\config.py, app\core\database.py,
```

## 3. Environment & DB Connection ( `app/core/database.py` )

In `.env` :

Code snippet

```
MONGO_URI=mongodb://localhost:27017
DB_NAME=bangla_akinator
```

In `app/core/database.py` :

Python

```python
import os
from motor.motor_asyncio import AsyncIOMotorClient
from dotenv import load_dotenv

load_dotenv()

MONGO_URI = os.getenv("MONGO_URI", "mongodb://localhost:27017")
DB_NAME = os.getenv("DB_NAME", "bangla_akinator")

client = AsyncIOMotorClient(MONGO_URI)
db = client[DB_NAME]

def get_database():
    return db
```

## 4. Populate Seed Data ( `scripts/seed_data.py` )

Bootstrap the database with a seed script containing 5–10 core Bangladeshi figures and baseline questions: PDF + 2

Python

```python
import asyncio
from motor.motor_asyncio import AsyncIOMotorClient

async def seed():
    client = AsyncIOMotorClient("mongodb://localhost:27017")
    db = client["bangla_akinator"]

    # Clear existing collections
    await db.questions.drop()
    await db.characters.drop()

    # 1. Insert Questions
    questions = [
        {"question_code": "q1", "text_bn": "চরিত্রটি কি একজন বাস্তব ব্যক্তি?", "active":
        {"question_code": "q2", "text_bn": "তিনি কি খেলাধুলার সাথে যুক্ত?", "active": T
        {"question_code": "q3", "text_bn": "তিনি কি একজন লেখক বা সাহিত্যিক?", "acti
        {"question_code": "q4", "text_bn": "তিনি কি নাটকে বা সিনেমায় অভিনয় করেছেন?"
        {"question_code": "q5", "text_bn": "তিনি কি একজন সংগীতশিল্পী?", "active": T:
    ]
    await db.questions.insert_many(questions)

    # 2. Insert Characters with Answer Mappings (1.0 = Yes, 0.0 = No, 0.5 = Unkno
    characters = [
        {
            "name_bn": "সাকিব আল হাসান",
            "category": "ক্রিকেটার",
            "image_url": "https://placehold.co/200x200?text=Shakib",
            "answers": {"q1": 1.0, "q2": 1.0, "q3": 0.0, "q4": 0.0, "q5": 0.0}
        },
        {
            "name_bn": "হুমায়ূন আহমেদ",
            "category": "লেখক",
            "image_url": "https://placehold.co/200x200?text=Humayun",
            "answers": {"q1": 1.0, "q2": 0.0, "q3": 1.0, "q4": 0.5, "q5": 0.0}
        },
        {
            "name_bn": "মোশাররফ করিম",
            "category": "অভিনেতা",
            "image_url": "https://placehold.co/200x200?text=Mosharraf",
            "answers": {"q1": 1.0, "q2": 0.0, "q3": 0.0, "q4": 1.0, "q5": 0.0}
        },
        {
            "name_bn": "তাহসান খান",
            "category": "গায়ক/অভিনেতা",
            "image_url": "https://placehold.co/200x200?text=Tahsan",
```

```
            "answers": {"q1": 1.0, "q2": 0.0, "q3": 0.0, "q4": 1.0, "q5": 1.0}
        }
    ]
    await db.characters.insert_many(characters)
    print("Database seeded successfully with initial Bangla characters and questi

if __name__ == "__main__":
    asyncio.run(seed())
```

Run the seed script:

PowerShell

```
python scripts\seed_data.py
```

Open MongoDB Compass, connect to `mongodb://localhost:27017`, and verify that the `bangla_akinator` database has both collections. PDF

## 4. Building Phase 2: FastAPI Core Game Engine

The game engine tracks user sessions, computes the question that closest approaches a 50/50 entropy split among the remaining pool, updates character scores, and decides when confidence is high enough to trigger a guess. PDF + 2

### 1. Data Schemas ( `app/api/schemas.py` )

Python

```
from pydantic import BaseModel
from typing import Dict, Optional, List

class StartGameResponse(BaseModel):
    session_id: str
    question_code: str
    text_bn: str
    question_number: int
```

```python
class AnswerRequest(BaseModel):
    session_id: str
    question_code: str
    # 1.0 = Yes, 0.0 = No, 0.5 = Don't Know, 0.75 = Probably, 0.25 = Probably Not
    answer_value: float

class AnswerResponse(BaseModel):
    status: str # "next_question", "guess", or "defeat"
    question_code: Optional[str] = None
    text_bn: Optional[str] = None
    question_number: Optional[int] = None
    character_guess: Optional[dict] = None

class AddCharacterRequest(BaseModel):
    session_id: str
    name_bn: str
    category: str
```

## 2. Shannon Entropy & Probability Logic ( `app/services/game_engine.py` )

Python

```python
import math
from typing import Dict, List, Any, Optional

class GameSession:
    def __init__(self, session_id: str, characters: List[Dict[str, Any]], questio
        self.session_id = session_id
        self.all_questions = {q["question_code"]: q["text_bn"] for q in questions
        self.asked_questions: List[str] = []
        self.user_answers: Dict[str, float] = {}
        self.question_count = 0

        # Initialize probabilities evenly across all characters
        total = len(characters)
        self.characters = characters
        self.probabilities: Dict[str, float] = {
            str(c["_id"]): 1.0 / total for c in characters
        }

    def pick_best_question(self) -> Optional[str]:
```

```
        """Finds the unasked question that splits active candidates closest to 50
        best_question = None
        min_distance_to_half = float("inf")

        unasked = [q for q in self.all_questions.keys() if q not in self.asked_qu
        if not unasked:
            return None

        # Filter characters that still have a reasonable probability
        active_candidates = [
            c for c in self.characters if self.probabilities[str(c["_id"])] > 0.0
        ]
        if not active_candidates:
            active_candidates = self.characters

        for q_code in unasked:
            yes_count = 0
            for c in active_candidates:
                val = c.get("answers", {}).get(q_code, 0.5)
                if val >= 0.75:
                    yes_count += 1

            # Target split is 50% of active pool
            distance = abs((len(active_candidates) / 2.0) - yes_count)
            if distance < min_distance_to_half:
                min_distance_to_half = distance
                best_question = q_code

        return best_question

    def record_answer(self, q_code: str, answer_val: float):
        self.asked_questions.append(q_code)
        self.user_answers[q_code] = answer_val
        self.question_count += 1

        # Bayes-style probability weight adjustment
        total_weight = 0.0
        for c in self.characters:
            c_id = str(c["_id"])
            expected = c.get("answers", {}).get(q_code, 0.5)
            # Distance between user response and expected character attribute
            diff = abs(expected - answer_val)
            multiplier = 1.0 - diff  # Exact match yields 1.0, complete mismatch

            self.probabilities[c_id] *= max(multiplier, 0.1)
            total_weight += self.probabilities[c_id]
```

```
        # Normalize probabilities to sum to 1.0
        if total_weight > 0:
            for c_id in self.probabilities:
                self.probabilities[c_id] /= total_weight

    def get_top_candidate(self):
        sorted_candidates = sorted(self.probabilities.items(), key=lambda x: x[1]
        top_id, score = sorted_candidates[0]
        top_char = next(c for c in self.characters if str(c["_id"]) == top_id)
        return top_char, score
```

## 3. API Router ( app/api/endpoints.py )

Python

```
import uuid
from fastapi import APIRouter, HTTPException
from app.core.database import get_database
from app.services.game_engine import GameSession
from app.api.schemas import StartGameResponse, AnswerRequest, AnswerResponse, Add

router = APIRouter()
sessions: dict[str, GameSession] = {}

@router.post("/start-game", response_model=StartGameResponse)
async def start_game():
    db = get_database()
    characters = await db.characters.find().to_list(1000)
    questions = await db.questions.find({"active": True}).to_list(1000)

    if not characters or not questions:
        raise HTTPException(status_code=500, detail="Database has not been seeded

    session_id = str(uuid.uuid4())
    session = GameSession(session_id, characters, questions)
    sessions[session_id] = session

    first_question = session.pick_best_question()
    session.asked_questions.append(first_question)
```

```
        return {
            "session_id": session_id,
            "question_code": first_question,
            "text_bn": session.all_questions[first_question],
            "question_number": 1
        }

    @router.post("/answer", response_model=AnswerResponse)
    async def submit_answer(req: AnswerRequest):
        session = sessions.get(req.session_id)
        if not session:
            raise HTTPException(status_code=404, detail="Session expired or invalid."

        session.record_answer(req.question_code, req.answer_value)
        top_char, confidence = session.get_top_candidate()

        # Trigger a guess if confidence > 75% or 15 questions have been asked
        if (confidence > 0.75 or session.question_count >= 15) and session.question_c
            return {
                "status": "guess",
                "character_guess": {
                    "name_bn": top_char["name_bn"],
                    "category": top_char["category"],
                    "image_url": top_char.get("image_url", ""),
                    "confidence": round(confidence * 100, 1)
                }
            }

        next_q = session.pick_best_question()
        if not next_q:
            # All available questions exhausted without a clear winner
            return {"status": "defeat"}

        session.asked_questions.append(next_q)
        return {
            "status": "next_question",
            "question_code": next_q,
            "text_bn": session.all_questions[next_q],
            "question_number": session.question_count + 1
        }

    @router.post("/add-character")
    async def add_character(req: AddCharacterRequest):
        session = sessions.get(req.session_id)
        if not session:
            raise HTTPException(status_code=404, detail="Invalid session.")
```

```
    db = get_database()
    # Backfill with the answers the user just gave during their game session
    new_char = {
        "name_bn": req.name_bn,
        "category": req.category,
        "image_url": "https://placehold.co/200x200?text=Custom",
        "answers": session.user_answers
    }
    await db.characters.insert_one(new_char)
    del sessions[req.session_id]
    return {"message": "Character added successfully! The genie learned from you.
```

## 4. Server Entry Point ( `app/main.py` )

Python

```python
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from app.api.endpoints import router

app = FastAPI(title="Bangla Akinator API")

# Allow Flutter development connections
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

app.include_router(router, prefix="/api")

@app.get("/health")
def health():
    return {"status": "ok"}
```

Run and test the backend:

PowerShell

```
uvicorn app.main:app --reload --port 8000
```

Open `http://localhost:8000/docs` in your browser. FastAPI's interactive Swagger UI lets you trigger `/api/start-game` and `/api/answer` directly to confirm the entropy calculations function before touching the frontend. PDF

## 5. Building Phase 3: Flutter Cross-Platform Frontend

Return to the root `bangla_akinator` folder:

PowerShell

```
cd ..
flutter create akinator_frontend
cd akinator_frontend
```

### 1. Add Dependencies ( `pubspec.yaml` )

Open `pubspec.yaml` and add:

YAML

```
dependencies:
  flutter:
    sdk: flutter
  http: ^1.2.0
  google_fonts: ^6.2.1
```

Run `flutter pub get` .

## 2. Network Client ( `lib/services/api_service.dart` )

On Windows or Web, `http://localhost:8000` points to your machine. For the Android emulator, use `[http://10.0.2.2:8000](http://10.0.2.2:8000)` .

Dart

```dart
import 'dart:convert';
import 'package:http/http.dart' as http;

class ApiService {
  // Use http://10.0.2.2:8000 for Android Emulator; localhost:8000 for Web/Window
  static const String baseUrl = "http://127.0.0.1:8000/api";

  Future<Map<String, dynamic>> startGame() async {
    final res = await http.post(Uri.parse('$baseUrl/start-game'));
    if (res.statusCode == 200) {
      return jsonDecode(utf8.decode(res.bodyBytes));
    }
    throw Exception("Failed to start game");
  }

  Future<Map<String, dynamic>> submitAnswer(String sessionId, String qCode, doubl
    final res = await http.post(
      Uri.parse('$baseUrl/answer'),
      headers: {'Content-Type': 'application/json'},
      body: jsonEncode({
        'session_id': sessionId,
        'question_code': qCode,
        'answer_value': answerValue,
      }),
    );
    if (res.statusCode == 200) {
      return jsonDecode(utf8.decode(res.bodyBytes));
    }
    throw Exception("Failed to submit answer");
  }

  Future<void> addCharacter(String sessionId, String nameBn, String category) asy
    final res = await http.post(
      Uri.parse('$baseUrl/add-character'),
      headers: {'Content-Type': 'application/json'},
      body: jsonEncode({
        'session_id': sessionId,
```

```
          'name_bn': nameBn,
          'category': category,
        }),
      );
      if (res.statusCode != 200) {
        throw Exception("Failed to save character");
      }
    }
  }
```

## 3. Core Game UI ( `lib/main.dart` )

Replace `lib/main.dart` with the primary game loop:

Dart

```dart
import 'package:flutter/material.dart';
import 'package:google_fonts/google_fonts.dart';
import 'services/api_service.dart';

void main() => runApp(const BanglaAkinatorApp());

class BanglaAkinatorApp extends StatelessWidget {
  const BanglaAkinatorApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'বাংলা জিন (Akinator)',
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.deepPurple),
        textTheme: GoogleFonts.hindSiliguriTextTheme(),
      ),
      home: const GameScreen(),
    );
  }
}

class GameScreen extends StatefulWidget {
  const GameScreen({super.key});
```

```
  @override
  State<GameScreen> createState() => _GameScreenState();
}

class _GameScreenState extends State<GameScreen> {
  final ApiService api = ApiService();
  String? sessionId;
  String currentQuestion = "খেলা শুরু করতে অপেক্ষা করুন...";
  String currentQCode = "";
  int questionIndex = 1;
  bool isLoading = true;
  Map<String, dynamic>? guessData;

  @override
  void initState() {
    super.initState();
    _start();
  }

  void _start() async {
    setState(() => isLoading = true);
    try {
      final data = await api.startGame();
      setState(() {
        sessionId = data['session_id'];
        currentQCode = data['question_code'];
        currentQuestion = data['text_bn'];
        questionIndex = data['question_number'];
        guessData = null;
        isLoading = false;
      });
    } catch (e) {
      setState(() {
        currentQuestion = "সার্ভারের সাথে সংযোগ বিচ্ছিন্ন। ব্যাকএন্ড চালু আছে কি না পরীক্ষা করু
        isLoading = false;
      });
    }
  }

  void _sendAnswer(double val) async {
    if (sessionId == null) return;
    setState(() => isLoading = true);
    final res = await api.submitAnswer(sessionId!, currentQCode, val);

    if (res['status'] == 'guess') {
      setState(() {
```

```
        guessData = res['character_guess'];
        isLoading = false;
      });
    } else if (res['status'] == 'defeat') {
      _showDefeatDialog();
    } else {
      setState(() {
        currentQCode = res['question_code'];
        currentQuestion = res['text_bn'];
        questionIndex = res['question_number'];
        isLoading = false;
      });
    }
  }

  void _showDefeatDialog() {
    final nameController = TextEditingController();
    final catController = TextEditingController();

    showDialog(
      context: context,
      barrierDismissible: false,
      builder: (ctx) => AlertDialog(
        title: const Text('আপনি জিতে গেছেন! জিন হেরে গেছে! 🤦'),
        content: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            const Text('আপনি কার কথা ভেবেছিলেন? নামটি শিখিয়ে দিন:'),
            TextField(controller: nameController, decoration: const InputDecorati
            TextField(controller: catController, decoration: const InputDecoratio
          ],
        ),
        actions: [
          ElevatedButton(
            onPressed: () async {
              if (nameController.text.isNotEmpty) {
                await api.addCharacter(sessionId!, nameController.text, catContro
              }
              Navigator.pop(ctx);
              _start();
            },
            child: const Text('সংরক্ষণ করুন ও আবার খেলুন'),
          )
        ],
      ),
    );
```

```
  }

  @override
  Widget build(BuildContext context) {
    if (isLoading) {
      return const Scaffold(body: Center(child: CircularProgressIndicator()));
    }

    if (guessData != null) {
      return Scaffold(
        appBar: AppBar(title: const Text('জিনের অনুমান!')),
        body: Center(
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              Text("আপনি কি এর কথা ভাবছিলেন?", style: Theme.of(context).textTheme
              const SizedBox(height: 16),
              Text(guessData!['name_bn'], style: const TextStyle(fontSize: 32, fo
              Text("ক্যাটাগরি: ${guessData!['category']}", style: const TextStyle(fc
              const SizedBox(height: 24),
              Row(
                mainAxisAlignment: MainAxisAlignment.center,
                children: [
                  ElevatedButton(onPressed: () => _start(), child: const Text("হ্যাঁ
                  const SizedBox(width: 16),
                  ElevatedButton(onPressed: () => _showDefeatDialog(), child: con
                ],
              )
            ],
          ),
        ),
      );
    }

    return Scaffold(
      appBar: AppBar(title: Text('প্রশ্ন নং $questionIndex')),
      body: Padding(
        padding: const EdgeInsets.all(24.0),
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          crossAxisAlignment: CrossAxisAlignment.stretch,
          children: [
            Container(
              padding: const EdgeInsets.all(20),
              decoration: BoxDecoration(color: Colors.purple.shade50, borderRadiu
              child: Text(
```

```
                currentQuestion,
                textAlign: TextAlign.center,
                style: const TextStyle(fontSize: 22, fontWeight: FontWeight.w600)
              ),
            ),
            const SizedBox(height: 30),
            _buildAnswerButton("হ্যাঁ (Yes)", 1.0, Colors.green),
            const SizedBox(height: 8),
            _buildAnswerButton("সম্ভবত হ্যাঁ (Probably)", 0.75, Colors.teal),
            const SizedBox(height: 8),
            _buildAnswerButton("জানি না (Don't Know)", 0.5, Colors.grey),
            const SizedBox(height: 8),
            _buildAnswerButton("সম্ভবত না (Probably Not)", 0.25, Colors.orange),
            const SizedBox(height: 8),
            _buildAnswerButton("না (No)", 0.0, Colors.red),
          ],
        ),
      ),
    );
  }

  Widget _buildAnswerButton(String label, double val, Color color) {
    return ElevatedButton(
      style: ElevatedButton.styleFrom(backgroundColor: color, foregroundColor: Co
      onPressed: () => _sendAnswer(val),
      child: Text(label, style: const TextStyle(fontSize: 16)),
    );
  }
}
```

## 6. Development Workflow & Running the App

To test your full stack locally:

1. **Start MongoDB:** Ensure the service is active or run `mongod` in a terminal.
2. **Start FastAPI Backend:**

PowerShell

```
cd bangla_akinator\akinator_backend
.\venv\Scripts\Activate.ps1
uvicorn app.main:app --reload --port 8000
```

3. **Run Flutter:**
   Open another terminal window:

PowerShell

```
cd bangla_akinator\akinator_frontend
flutter run -d chrome
```